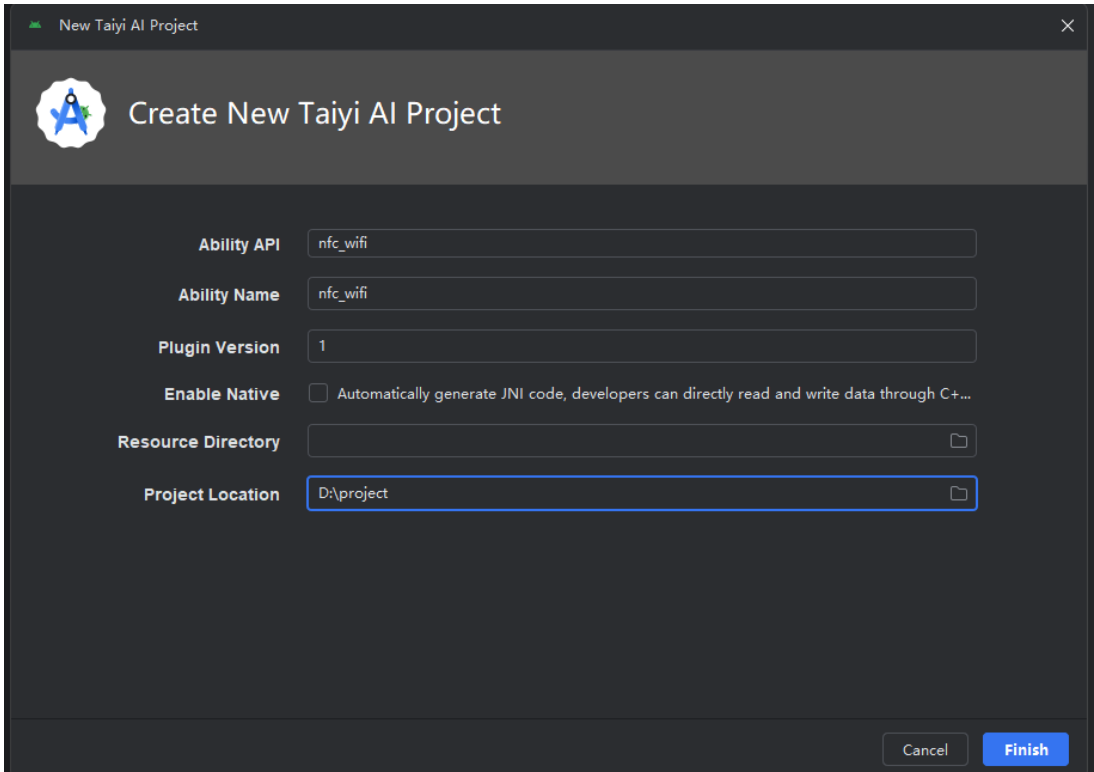
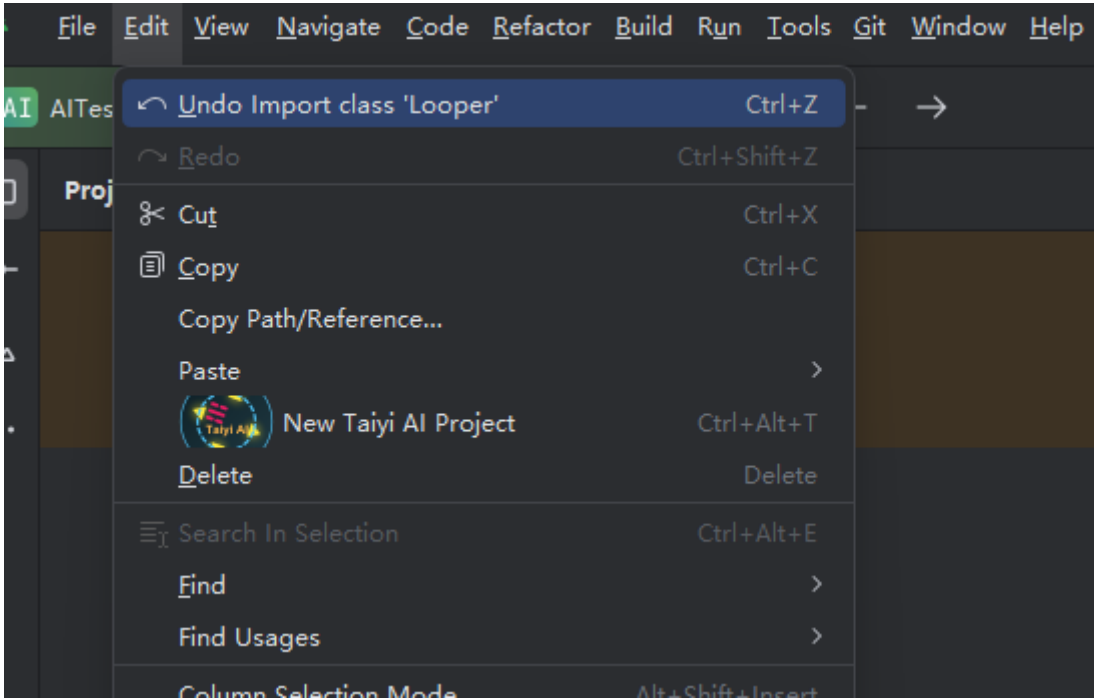


nfc_wifi插件开发

1.项目创建



2.查看需要封装的接口



- 这个是需要算法提供的，最好让他写详细一点，参数及返回值有什么意义，函数的作用是什么，方便自己理解。
- 确定需要的资源有哪些，这个算法不需要依赖模型，只需要封装两个接口，也是询问过后确定自己实现算法方给的两个接口就好，在他写的demo里，这两个接口也确实没有依赖其他的包

太一平台-NFC智能选卡优化项目

- 需要封装的共有两个函数：

代码块

```
1     private data class RawSensorRecord(  
2         val timestamp: Long,    // 时间戳  
3         val rawLine: String,    // 原始行数据  
4         val macKey: String      // MAC地址分组键  
5     )
```

- cleanRawRecords

代码块

```
1  //输入的是原始数据库的数据，下面将对其中的数据进行筛选，以避免数据库无限扩大，筛选主要包含  
2  //以下三个步骤：  
3  //1. 丢弃时间戳超过当前maxDays的数据；  
4  //2. 对于MAC 地址相同的数据，按规则进行处理；  
5  //3. 若数据量超过maxTotalRecords，则保留时间最近的maxTotalRecords条数据；  
6  //返回的是筛选过后的原始数据，用于覆盖原始数据库。  
7  private fun cleanRawRecords(  
8      rawRecords: List<RawSensorRecord>,  
9      maxDays: Int = 30,  
10     maxTotalRecords: Int = 10000,  
11     maxIntervalForLabelMatch: Float = 10f  
12 ): List<RawSensorRecord> {  
13     val currentTime = System.currentTimeMillis()  
14     val maxAgeMillis = maxDays * 24 * 3600 * 1000L  
15     // 新增解析结构  
16     data class ParsedRecord(  
17         val timestamp: Long,  
18         val label: Float,  
19         val interval: Float,  
20         val signal: Float,  
21         val mac: FloatArray,  
22         val rawLine: String
```

```

23     )
24
25     // 1 全局解析原始数据
26     val parsedList = rawRecords
27         .mapNotNull { record ->
28             val parts = record.rawLine.split(',')
29             if(parts.size != 18) return@mapNotNull null
30
31             try {
32                 ParsedRecord(
33                     timestamp = parts[0].toLong(),
34                     label = parts[1].toFloat(),
35                     interval = parts[3].toFloat(),
36                     signal = parts[4].toFloat(),
37                     mac = parts.subList(5, 13).map { it.toFloat()
38 }.toFloatArray(), // 8个MAC字段
39                     rawLine = record.rawLine
40                 )
41             } catch(e: Exception) {
42                 null
43             }
44         }.filter { currentTime - it.timestamp <= maxAgeMillis }
45
46     // 2 MAC地址分组处理
47     val groupedByMac = parsedList.groupBy { it.mac.joinToString(",") {
48         "%.1f".format(it) } }
49
50     val processedRecords = mutableListOf<ParsedRecord>()
51
52     for((mac, group) in groupedByMac) {
53         // 3 小间隔数据处理
54         val smallGroup = group.filter { it.interval < maxIntervalForLabelMatch
55 }
56
57         val intervalTimestampThre = smallGroup.maxOfOrNull { it.timestamp } ?:
58         0L
59
60         // 保留small组每个标签的最新记录
61         val smallProcessed = smallGroup
62             .groupBy { it.label }
63             .mapValues { entry ->
64                 entry.value.maxByOrNull { it.timestamp }
65             }
66
67         .values.filterNotNull()
68
69         // 4 大间隔数据处理
70         val bigGroup = group.filter {
71             it.interval >= maxIntervalForLabelMatch &&

```

```

66             it.timestamp >= intervalTimestampThre
67         }
68
69         // 按优先级排序: signal降序 -> interval升序 -> timestamp降序
70         val sortedBigGroup = bigGroup.sortedWith(
71             compareByDescending<ParsedRecord> { it.signal }
72                 .thenBy { it.interval }
73                 .thenByDescending { it.timestamp }
74         )
75
76         // 保留高优先级数据 (时间戳必须递增)
77         val bigProcessed = mutableListOf<ParsedRecord>()
78         var currentMaxTimestamp = 0L
79
80         for(record in sortedBigGroup) {
81             if(record.timestamp > currentMaxTimestamp) {
82                 bigProcessed.add(record)
83                 currentMaxTimestamp = record.timestamp
84             }
85         }
86
87         processedRecords.addAll(smallProcessed)
88         processedRecords.addAll(bigProcessed)
89     }
90
91     // 5 总量控制
92     return processedRecords
93         .sortedByDescending { it.timestamp }
94         .take(maxTotalRecords)
95         .map { RawSensorRecord(it.timestamp, it.rawLine,
96             it.mac.joinToString(", ")) }

```

- buildMatchList

代码块

```

1 //输入是通过过滤的原始数据库的数据 (records) , macLength表示mac地址数据的长度,
  // maxIntervalForLabelMatch表示高置信度数据的阈值
2 //具体处理过程:
3 //1. 所有数据先按照mac地址进行分组。
4 //2. 若同组数据中, 存在interval < maxIntervalForLabelMatch的数据:
5 //    2.1 若这些数据对应的标签唯一, 则此标签即为该mac地址学习到的映射
6 //    2.2 若这些数据对应的标签不唯一, 则该mac地址无对应的标签
7 //3. 若同组数据中, 所有数据的interval >= maxIntervalForLabelMatch, 则按照下面的规则进
  // 行排序

```

```

8 // 3.1 signal大的靠前
9 // 3.2 interval 小的靠前
10 // 3.3 timestamp 大的靠前
11 // 3.4 取排序第一的作为该mac地址 对应的标签
12 //输出是学习到的映射关系
13 private fun buildMatchList(
14     records: List<RawSensorRecord>,
15     macLength: Int = 8,
16     maxIntervalForLabelMatch: Float = 10f
17 ): List<FloatArray> {
18     val result = mutableListOf<FloatArray>()
19
20     val grouped = records.groupBy { it.macKey }
21
22     grouped.values.forEach { group ->
23         // 解析时使用rawLine属性
24         val groupRecords = group.map { rawRecord ->
25             val parts = rawRecord.rawLine.split(',').map { it.toFloatOrNull()
26                 ?: 0f }
27             SensorRecord(
28                 timestamp = rawRecord.timestamp,
29                 label = parts[1],
30                 interval = parts[3],
31                 signal = parts[4],
32                 mac = FloatArray(macLength) { i -> parts[5 + i] }
33             )
34         }
35         val lowIntervalRecords = groupRecords.filter { it.interval <=
36             maxIntervalForLabelMatch }
37         if (lowIntervalRecords.isNotEmpty()) {
38             val labels = lowIntervalRecords.map { it.label }
39             if (labels.distinct().size == 1) {
40                 result.add(floatArrayOf(labels.first(),
41                     *groupRecords.first().mac))
42             }
43         } else {
44             val best = groupRecords.sortedWith(
45                 compareByDescending<SensorRecord> { it.signal }
46                     .thenBy { it.interval }
47                     .thenByDescending { it.timestamp }
48             ).first()
49             result.add(floatArrayOf(best.label, *best.mac))
50         }
51     }
52     return result
53 }

```

3.确定序列化/反序列化的方式

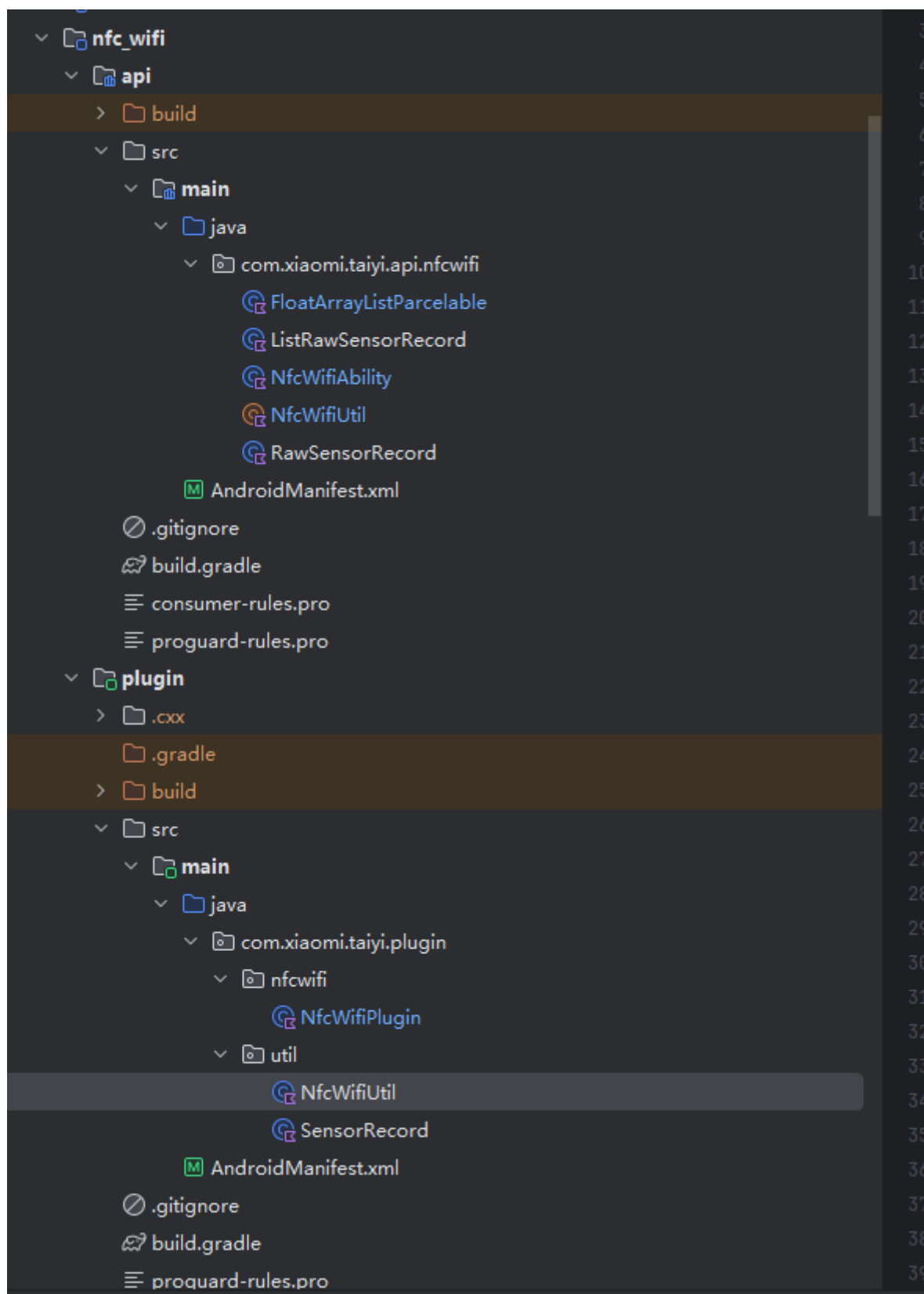


- 要询问算法方，传输的数据量的大小
- 如果数据量超过 1KB， 不要使用 Json， 1 KB 以内使用 Json 的兼容性、普适性更好。
- 如果数据量在 100KB 以内，使用 Parcelable， binder 通信（不超过 1MB），效率更高。
- 如果在 100KB 以上，建议使用 Protocolbuf 执行 ShareMemory 共享内存通信。
- 如果数据量小于 100KB ，可以使用FlatBuffer。如果使用flatBuffer,应注意他的数据类型不能和其他方法共享。

以paceable为例，序列化后都是字节数组格式，适合使用putData和getData函数（具体选择可以去AIPackage和AIBody查看）

4.开发

1.总体结构



2.接口的封装

如 **NfcWifiUtil** 所示，其实是直接生成了一个类，把他给的函数放进去了，SensorRecord是函数中要用到的一个数据类，如果需要依赖jar包的话，可以直接在plugin目录下加一个libs目录，把jar包放进去，然后在plugin的bulid.gradle中添加依赖。

3.NfcWifiPlugin



这里api和plugin在逻辑上的关系就是C/S模式，api(客户端)通过创建request请求plugin(服务器)，并返回response。

具体实现是在api和plugin中会有个中间层即service层，请参考 [【太一 AI 平台】插件开发指南](#)

process是插件的入口函数，在process函数中根据方法名执行具体函数

```
Process

1  override fun process(request: AIRequest, response: AIResponse) {
2      super.process(request, response)
3      val method = request.headers.getMethod()
4      logger.i(TAG, "process: ${method}")
5
6      when (request.headers.getMethod()) {
7          METHOD_NFCWIFI_CLEANRAWRECORDS -> {
8              logger.i(TAG, "process: cleanRawRecords")
9              cleanRawRecords(request, response)
10         }
11
12         METHOD_NFCWIFI_BUILDMATCHLIST -> {
13             logger.i(TAG, "process: buildMatchList")
14             buildMatchList(request, response)
15         }
16
17         else -> {
18             logger.i(TAG, "process: no such method")
19             response.code = ErrorCode.ERROR_PROCESS_FAILED
20             response.body.setSpecCode(NFC_NO_SUCH_METHOD)
21         }
22     }
23
24 }
```

下面以一个具体实现为例，讲解需要注意的事项

1. 打印日志的工具logger，因为在api中打印日志需要SdkLog，方便查询
2. 在类型转换的时候使用as?(失败时返回null，避免抛出异常)替换 as
3. 在参数验证中，可以注意一下格式，多个表达式带上括号，操作符放到左侧
4. 错误码的使用，执行错误用FAILED，捕获异常中使用CRASH，数据读取为空使用EMPTY等等，去 ErrorCode中查看。

5. 大家可以参考这个顺序，1.提取参数2. 参数验证3. 执行4. 返回结果

```
cleanRawRecords

1  fun cleanRawRecords(request: AIRequest, response: AIResponse) {
2      logger.d(TAG, "cleanRawRecords: start processing")
3
4      try {
5          // 1. 提取参数
6          val rawRecords = request.body.getData("rawRecords")
7          val rawRecordsParcelable = deserialize(rawRecords, false) as?
ListRawSensorRecord
8          val maxDays = request.body.getInt("maxDays", -1)
9          val maxTotalRecords = request.body.getInt("maxTotalRecords", -1)
10         val maxPerMac = request.body.getFloat("maxPerMac", -1f)
11
12         // 2. 参数验证
13         if ((rawRecordsParcelable == null)
14             || (rawRecordsParcelable.records.isEmpty())
15             || (maxDays < 0)
16             || (maxTotalRecords < 0)
17             || (maxPerMac < 0)
18         ) {
19             logger.w(TAG, "cleanRawRecords: invalid parameters")
20             response.code = ErrorCode.ERROR_PROCESS_FAILED
21             response.body.setSpecCode(NFC_INVALID_PARAM)
22             return
23         }
24
25         // 3. 执行
26         val result = util.cleanRawRecords(
27             rawRecordsParcelable.records,
28             maxDays,
29             maxTotalRecords,
30             maxPerMac
31         )
32
33         logger.d(
34             TAG,
35             "cleanRawRecords: processed ${rawRecordsParcelable.records.size} -
> ${result.size} records"
36         )
37
38         // 4. 返回结果
39         val resultParcelable = ListRawSensorRecord(result)
40         response.body.putData("result", serialize(resultParcelable, false) ?:
ByteArray(0))
```

```

41
42     } catch (e: Exception) {
43         logger.e(TAG, "cleanRawRecords: unexpected error, exception:
    ${e.message}")
44         response.code = ErrorCode.ERROR_PROCESS_CRASH
45         response.body.setSpecCode(NFC_PROCESS_ERROR)
46     }
47 }

```

4.plugin-build.gradle

代码块

```

1  dependencies {
2      compileOnly "com.xiaomi.taiyi.sdk:plugin:$version_taiyi_ai_sdk"
3      implementation project(":nfc_wifi:api")
4  }

```

在序列化/反序列化中，由包名和类名确定类，也就是说尽管你在api和plugin中同时实现了两个相同和类，但是在反序列化时依然会失败，所以我们将定义的继承parcelable的类都放到api中，让plugin依赖api。

5.plugin-proguard-rules.pro

可以先复制一份别人比较全的。

一般和自己相关的是：

代码块

```

1  # 插件相关
2  -repackageclasses com.xiaomi.taiyi.plugin.nfcwifi
3  -keep class com.xiaomi.taiyi.plugin.nfcwifi.** { *; }
4  -keep class com.xiaomi.taiyi.plugin.nfcwifi.**$* { *; }

```

1. -repackageclasses com.xiaomi.taiyi.plugin.nfcwifi

- **作用：**将所有经过混淆处理的类（除了被 `-keep` 保留的类）重新打包到指定的包名下（这里是 `com.xiaomi.taiyi.plugin.nfcwifi`）。
- **效果：**
 - 减少应用的包数量，使类结构更紧凑。
 - 进一步隐藏原始代码的包结构，增强混淆效果。
- **注意：**被 `-keep` 保留的类不会被移动，仅未被保留的混淆类会被重新打包。

2. `-keep class com.xiaomi.taiyi.plugin.nfcwifi.** { *; }`

- **作用：**保留指定包下的所有类及其成员（字段和方法）不被混淆。
 - `com.xiaomi.taiyi.plugin.nfcwifi.**` 表示匹配该包下的所有类（包括子包中的类）。
 - `{ *; }` 表示保留类中的所有字段和方法（无论访问权限如何）。
- **适用场景：**
 - 该包下的类需要被外部（如反射、跨模块调用）访问，若被混淆会导致功能异常。
 - 避免核心逻辑类的名称和结构被篡改。

3. `-keep class com.xiaomi.taiyi.plugin.nfcwifi.**$* { *; }`

- **作用：**保留指定包下所有类的内部类（包括匿名内部类、静态内部类等）及其成员不被混淆。
 - `**$*` 中的 `$` 是 Java/Kotlin 中内部类的命名分隔符（如 `OuterClass$InnerClass`）。
- **必要性：**
 - 内部类通常与外部类有紧密依赖，若被混淆可能导致内部逻辑调用失败。
 - 尤其适用于通过反射访问内部类的场景。

6.api-NfcWifiAbility



api是暴露给业务的接口，主要就是将输入封装进request，传递给插件，然后处理响应

1. 使用SdkLog
2. 尽量不要抛出异常，可以直接返回空值，这样就可以让业务方先去纠错
3. 也可以参考一下实现的顺序

代码块

```
1  @JvmOverloads
2  fun cleanRawRecords(
3      rawRecords: List<RawSensorRecord>,
4      maxDays: Int = 30,
5      maxTotalRecords: Int = 10000,
6      maxPerMac: Float = 10f
7  ): List<RawSensorRecord> {
8      // 1. 输入验证
9      if (rawRecords.isEmpty()) {
10         SdkLog.d(TAG, "cleanRawRecords: input list is empty")
11         return emptyList()
12     }
```

```

13
14 // 2. 构建请求
15 val request = createRequest()
16 try {
17     val rawRecordsParcelable = ListRawSensorRecord(rawRecords)
18     val serializerRecords = serialize(rawRecordsParcelable, false)
19     if (serializerRecords == null) {
20         SdkLog.e(TAG, "cleanRawRecords: failed to serialize records")
21         return emptyList()
22     }
23     request.headers.setMethod("cleanRawRecords")
24     request.body.apply {
25         putData("rawRecords", serializerRecords)
26         putInt("maxDays", maxDays)
27         putInt("maxTotalRecords", maxTotalRecords)
28         putFloat("maxPerMac", maxPerMac)
29     }
30 } catch (e: Exception) {
31     SdkLog.e(TAG, "cleanRawRecords: failed to build request. ${e.message}")
32     return emptyList()
33 }
34
35 // 3. 发送请求
36 val response = try {
37     execute(request)
38 } catch (e: Exception) {
39     SdkLog.e(TAG, "cleanRawRecords: network request failed. ${e.message}")
40     return emptyList()
41 }
42
43 // 4. 处理响应
44 val errorCode = response.body.getErrorCode()
45 SdkLog.d(TAG, "cleanRawRecords: response errorCode=$errorCode")
46
47 val result = response.body.getData("result")
48 if (result == null) {
49     SdkLog.e(TAG, "cleanRawRecords: result bundle is null")
50     return emptyList()
51 }
52
53 return (deserialize(result, false) as? ListRawSensorRecord)?.records ?:
    emptyList()
54 }

```

涉及跨进程通信或序列化的类需要在这里配置

plugin不需要这个，可以直接删掉这个文件

代码块

```
1  -keep class com.xiaomi.taiyi.api.nfcwifi.** { public *; }
2  -keep class com.xiaomi.taiyi.api.nfcwifi.**$* { public *; }
```

8.api-proguard-rules.pro

也是可以先复制一份别人的

注意由于我们使用的是parcelable序列化方式，所以需要：

代码块

```
1  # 保留 Parcelable 实现类
2  -keep class * implements android.os.Parcelable {
3      public static final android.os.Parcelable$Creator *;
4  }
```

再有其他相关的就是：

代码块

```
1  -keepparameternames
2  -keep class com.xiaomi.taiyi.api.nfcwifi.** { public *; }
3  -keep class com.xiaomi.taiyi.api.nfcwifi.**$* { public *; }
```

9.项目的settings.gradle

代码块

```
1  include(':nfc_wifi:api')
2  include(':nfc_wifi:plugin')
```

10.app-build.gradle

代码块

```
1  implementation project(":nfc_wifi:api")
```

